

Multilayer Perceptron — Mathematical Foundations

Project Evaluation Document

Quick Reference: Backward pass, then Forward pass

Backward pass — for $\ell = L, L - 1, \dots, 1$, given the upstream gradient $\partial\mathcal{L}/\partial\mathbf{a}^{(\ell)}$:

$$\Delta^{(\ell)} = \underbrace{\frac{\partial\mathcal{L}}{\partial\mathbf{a}^{(\ell)}}}_{\text{upstream gradient}} \odot \phi'(\underbrace{\mathbf{Z}^{(\ell)}}_{\text{weighted sum}})$$

$$\frac{\partial\mathcal{L}}{\partial\mathbf{W}^{(\ell)}} = (\mathbf{A}^{(\ell-1)})^T \Delta^{(\ell)}$$

$$\frac{\partial\mathcal{L}}{\partial\mathbf{b}^{(\ell)}} = \sum_{i=1}^B \Delta_i^{(\ell)}$$

$$\frac{\partial\mathcal{L}}{\partial\mathbf{a}^{(\ell-1)}} = \Delta^{(\ell)} (\mathbf{W}^{(\ell)})^T$$

Forward pass — for $\ell = 1, \dots, L$:

$$\mathbf{Z}^{(\ell)} = \mathbf{A}^{(\ell-1)} \mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)}$$

$$\mathbf{A}^{(\ell)} = \phi^{(\ell)}(\mathbf{Z}^{(\ell)})$$

Contents

1	Feedforward Propagation	4
1.1	Single Neuron	4
1.2	Layer Forward Pass	4
1.3	Example: $n = 3 \rightarrow m = 2$ (Single Sample)	4
1.4	Batch Mode	5
1.5	Network Forward Pass	5
2	Activation Functions	6
2.1	Sigmoid	6
2.2	Hyperbolic Tangent	6
2.3	ReLU (Rectified Linear Unit)	6
2.4	Softmax	6
2.5	Implementation (per-layer backward)	6
3	Loss Functions	7
3.1	Mean Squared Error (MSE)	7
3.2	Binary Cross-Entropy (BCE) (mandatory)	7
3.3	Categorical Cross-Entropy (CCE) (mandatory)	7
3.4	Implementation Notes	7
4	Data Pipeline	7
4.1	Loading and Label Encoding	8
4.2	Standardization (z-score)	8
4.3	Train/Validation Split	8
4.4	Model Persistence and Prediction	8
5	Backpropagation	9
5.1	Chain Rule — Scalar Refresher	9
5.2	The Three Core Gradients of One Layer	9
5.3	Summary of One Layer	12
5.4	Full Network Algorithm	12
5.5	Concrete Numerical Example	13
5.6	Correlation to Rust Code	15
5.7	Intuition in Natural Language	16
6	Gradient Descent	16
6.1	Stochastic Gradient Descent (SGD) (mandatory)	17
6.2	Adam (Adaptive Moment Estimation) (bonus)	17
6.3	Weight Initialization	18
6.4	Overall Training Phase (mandatory)	18
7	Overfitting	19
7.1	Definition	19
7.2	Why It Happens	19
7.3	Early Stopping — available in this project	19
7.4	Detection in Practice	20
7.5	This Project's Architecture	20

8	Evaluation Metrics	20
8.1	Turning predictions into labels	20
8.2	Accuracy (mandatory)	20
8.3	Precision with macro-averaging (bonus)	21
8.4	Performance testing note	21
9	Code Structure and Size	21
9.1	Per-file summary	23
9.2	Per-function breakdown	23
A	Notation Reference	23

1 Feedforward Propagation

1.1 Single Neuron

A perceptron computes a weighted sum of its inputs followed by a non-linear activation:

$$z = \sum_{i=1}^n w_i x_i + b = \mathbf{W}^T \mathbf{x} + b \quad a = \sigma(z)$$

where $\mathbf{x} \in \mathbb{R}^n$ is the input, $\mathbf{W} \in \mathbb{R}^n$ the weight vector, $b \in \mathbb{R}$ the bias, and σ an activation function.

1.2 Layer Forward Pass

For a dense layer with n inputs and m outputs:

$$\mathbf{z} = \mathbf{x}\mathbf{W} + \mathbf{b} \quad \mathbf{a} = \phi(\mathbf{z})$$

where $\mathbf{x} \in \mathbb{R}^{1 \times n}$, $\mathbf{W} \in \mathbb{R}^{n \times m}$, $\mathbf{b} \in \mathbb{R}^{1 \times m}$, $\mathbf{z} \in \mathbb{R}^{1 \times m}$.

The matrix multiplication contracts the input dimension n into the output dimension m :

$$\underbrace{\mathbf{z}}_{1 \times m} = \underbrace{\mathbf{x}}_{1 \times n} \underbrace{\mathbf{W}}_{n \times m} + \underbrace{\mathbf{b}}_{1 \times m}$$

Each element z_j is the dot product of the input vector with the j -th column of $\mathbf{W}$:

$$z_j = \sum_{i=1}^n x_i W_{ij} + b_j$$

Remark 1.1 (Row-vector vs column-vector convention). *Two conventions appear in the literature:*

$$\begin{array}{ll} \text{(row)} & \mathbf{z} = \mathbf{x}\mathbf{W} + \mathbf{b} & \mathbf{x} \in \mathbb{R}^{1 \times n}, \mathbf{W} \in \mathbb{R}^{n \times m} \\ \text{(col)} & \mathbf{z} = \mathbf{W}^T \mathbf{x} + \mathbf{b} & \mathbf{x} \in \mathbb{R}^{n \times 1}, \mathbf{W} \in \mathbb{R}^{n \times m} \end{array}$$

Both compute the same dot products $z_j = \sum_i x_i W_{ij} + b_j$. We adopt the row-vector convention because:

- 1. It extends naturally to batches: $\mathbf{X} \in \mathbb{R}^{B \times n}$ multiplied on the right by $\mathbf{W}$ keeps samples as rows.*
- 2. It avoids a transpose on the weight matrix in the forward pass.*
- 3. It matches the NumPy/TensorFlow convention where $\mathbf{X} @ \mathbf{W} + \mathbf{b}$ is the layer computation.*

1.3 Example: $n = 3 \rightarrow m = 2$ (Single Sample)

$$\mathbf{x} = [0.5 \quad 2.0 \quad 1.0], \quad \mathbf{W} = \begin{bmatrix} 0.1 & 0.4 \\ 0.3 & 0.2 \\ 0.5 & 0.6 \end{bmatrix}, \quad \mathbf{b} = [0.1 \quad 0.2]$$

Step 1: $\mathbf{xW}$ — each column of $\mathbf{W}$ is dotted with $\mathbf{x}$:

$$\begin{aligned}\mathbf{xW} &= \begin{bmatrix} 0.5 & 2.0 & 1.0 \end{bmatrix} \begin{bmatrix} 0.1 & 0.4 \\ 0.3 & 0.2 \\ 0.5 & 0.6 \end{bmatrix} \\ &= \begin{bmatrix} 0.5 \times 0.1 + 2.0 \times 0.3 + 1.0 \times 0.5 & 0.5 \times 0.4 + 2.0 \times 0.2 + 1.0 \times 0.6 \end{bmatrix} \\ &= \begin{bmatrix} 0.05 + 0.6 + 0.5 & 0.2 + 0.4 + 0.6 \end{bmatrix} = \begin{bmatrix} 1.15 & 1.2 \end{bmatrix}\end{aligned}$$

The inner dimension $n = 3$ is summed over and disappears — this is the dimensionality reduction.

Each weight W_{ij} connects input neuron i to output neuron j .

$$\begin{aligned}z_1 &= 0.5 \times 0.1 + 2.0 \times 0.3 + 1.0 \times 0.5 + 0.1 = 1.25 \\ z_2 &= 0.5 \times 0.4 + 2.0 \times 0.2 + 1.0 \times 0.6 + 0.2 = 1.40\end{aligned}$$

Step 2: Add bias and apply activation:

$$\begin{aligned}\mathbf{z} &= \begin{bmatrix} 1.15 & 1.2 \end{bmatrix} + \begin{bmatrix} 0.1 & 0.2 \end{bmatrix} = \begin{bmatrix} 1.25 & 1.4 \end{bmatrix} \\ \mathbf{a} = \phi(\mathbf{z}) &= [\sigma(1.25) \quad \sigma(1.4)] \approx \begin{bmatrix} 0.777 & 0.802 \end{bmatrix}\end{aligned}$$

1.4 Batch Mode

With B samples as rows of $\mathbf{X} \in \mathbb{R}^{B \times n}$:

$$\underbrace{\mathbf{Z}}_{B \times m} = \underbrace{\mathbf{X}}_{B \times n} \underbrace{\mathbf{W}}_{n \times m} + \underbrace{\mathbf{b}}_{1 \times m}$$

The bias is broadcast (added to every row). Example with $B = 2$:

$$\begin{aligned}\mathbf{X} &= \begin{bmatrix} 0.5 & 2.0 & 1.0 \\ 1.0 & 0.5 & 1.5 \end{bmatrix} \\ \mathbf{XW} &= \begin{bmatrix} 0.5 & 2.0 & 1.0 \\ 1.0 & 0.5 & 1.5 \end{bmatrix} \begin{bmatrix} 0.1 & 0.4 \\ 0.3 & 0.2 \\ 0.5 & 0.6 \end{bmatrix} = \begin{bmatrix} 1.15 & 1.2 \\ 1.0 & 1.4 \end{bmatrix} \\ \mathbf{Z} &= \begin{bmatrix} 1.15 & 1.2 \\ 1.0 & 1.4 \end{bmatrix} + \begin{bmatrix} 0.1 & 0.2 \end{bmatrix} = \begin{bmatrix} 1.25 & 1.4 \\ 1.1 & 1.6 \end{bmatrix}\end{aligned}$$

1.5 Network Forward Pass

An MLP chains L such layers:

$$\begin{aligned}\mathbf{a}^{(0)} &= \mathbf{x} \\ \mathbf{z}^{(\ell)} &= \mathbf{a}^{(\ell-1)} \mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)} \quad \ell = 1, \dots, L \\ \mathbf{a}^{(\ell)} &= \phi^{(\ell)}(\mathbf{z}^{(\ell)})\end{aligned}$$

Each layer projects from $n^{(\ell-1)}$ to $n^{(\ell)}$. The final output $\hat{\mathbf{y}} = \mathbf{a}^{(L)}$ is the prediction.

2 Activation Functions

2.1 Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad \sigma'(x) = \sigma(x)(1 - \sigma(x))$$

Output range $(0, 1)$ — useful for binary classification output. Prone to vanishing gradient for large $|x|$.

2.2 Hyperbolic Tangent

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad \tanh'(x) = 1 - \tanh^2(x)$$

Output range $(-1, 1)$ — zero-centered, mitigates some vanishing gradient issues compared to sigmoid.

2.3 ReLU (Rectified Linear Unit)

$$\text{ReLU}(x) = \max(0, x) \quad \text{ReLU}'(x) = \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases}$$

Computationally efficient, alleviates vanishing gradient for positive inputs. Can suffer from “dying ReLU” when neurons get stuck at zero.

2.4 Softmax

$$\text{softmax}(\mathbf{z})_i = \frac{e^{z_i - M}}{\sum_{j=1}^K e^{z_j - M}} \quad M = \max(z_1, \dots, z_K)$$

Converts logits $\mathbf{z} \in \mathbb{R}^K$ into a probability distribution over K classes. Subtracting the maximum $M = \max_j z_j$ (the *log-sum-exp trick*) ensures numerical stability: without it, large logits like e^{100} overflow, while $e^{z_i - M} \leq e^0 = 1$ keeps all terms bounded. The subtraction does not change the output because $\frac{e^{z_i - M}}{\sum e^{z_j - M}} = \frac{e^{z_i} e^{-M}}{\sum e^{z_j} e^{-M}} = \frac{e^{z_i}}{\sum e^{z_j}}$.

The Jacobian of softmax is:

$$\frac{\partial \text{softmax}(\mathbf{z})_i}{\partial z_j} = \begin{cases} p_i(1 - p_i) & i = j \\ -p_i p_j & i \neq j \end{cases}$$

Used in the output layer with categorical cross-entropy; the combination simplifies the gradient to $\mathbf{p} - \mathbf{y}_{\text{one-hot}}$.

2.5 Implementation (per-layer backward)

Each layer’s backward method applies the chain rule:

$$\underbrace{\frac{\partial \mathcal{L}}{\partial \mathbf{z}}}_{\text{upstream}} = \underbrace{\frac{\partial \mathcal{L}}{\partial \mathbf{a}}}_{\text{incoming grad}} \odot \underbrace{\phi'(\mathbf{z})}_{\text{activation derivative}}$$

where $\odot$ denotes element-wise multiplication. The result $\frac{\partial \mathcal{L}}{\partial \mathbf{z}}$ is passed on for weight gradient computation.

3 Loss Functions

Per the project subject, **binary cross-entropy** is the mandatory loss: the prediction program must evaluate its results with the binary cross-entropy error function, and the softmax output layer is required. *Categorical* cross-entropy is the multiclass generalization of BCE (the two coincide for two classes) and is what the CLI actually trains with. **MSE** is also implemented (for regression), but is neither mandatory nor one of the five bonus features listed in the subject.

3.1 Mean Squared Error (MSE)

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2 \quad \frac{\partial \mathcal{L}}{\partial \hat{y}_i} = \frac{2}{N} (\hat{y}_i - y_i)$$

where N is the total number of elements (batch size $\times$ output dim). Used for regression.

3.2 Binary Cross-Entropy (BCE) (mandatory)

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^N \left[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i) \right]$$

where $y_i \in \{0, 1\}$ and $\hat{y}_i \in (0, 1)$ (sigmoid output). Gradient:

$$\frac{\partial \mathcal{L}}{\partial \hat{y}_i} = \frac{1}{N} \frac{\hat{y}_i - y_i}{\hat{y}_i(1 - \hat{y}_i)}$$

3.3 Categorical Cross-Entropy (CCE) (mandatory)

For $K > 1$ classes with softmax output:

$$\mathcal{L}_{\text{CCE}} = -\frac{1}{N} \sum_{i=1}^N \log(\hat{y}_{i,c_i})$$

where c_i is the correct class index for sample i , and $\hat{y}_{i,c_i}$ is the predicted probability for that class. Gradient:

$$\frac{\partial \mathcal{L}}{\partial \hat{\mathbf{y}}_i} = \hat{\mathbf{y}}_i - \mathbf{e}_{c_i}$$

where $\mathbf{e}_{c_i}$ is the one-hot vector. This elegant gradient (softmax output minus one-hot label) is why CCE + softmax are used together.

3.4 Implementation Notes

All implementations clamp predictions to $[\varepsilon, 1 - \varepsilon]$ with $\varepsilon = 10^{-12}$ to prevent $\log(0)$. The gradient computation reuses intermediate values from the forward pass for efficiency.

4 Data Pipeline

Before any learning happens, the raw CSV is turned into numeric tensors. The subject's mandatory workflow — train on `data_training.csv`, then evaluate the learned model on `data_test.csv` — requires the exact same preprocessing to be applied to both files, and this is what the project does: the scaler statistics are fitted on the training file alone and reused verbatim on the test file. This section covers loading and label encoding (Section 4.1), standardization (Section 4.2), the train/validation split (Section 4.3) and the save/load/predict loop (Section 4.4).

4.1 Loading and Label Encoding

The CSV is read by `load_dataset (src/data/loader.rs)` with Polars. Numeric columns are cast to `f64`, with missing values filled by `0.0`. String columns are *label-encoded*: the distinct classes are sorted alphabetically and each cell is replaced by its rank in that order. For the breast-cancer dataset the `Diagnosis` column therefore becomes 0 for “B” (benign) and 1 for “M” (malignant). The encoding is deterministic — it depends on the sorted class order, not on the row order — so a prediction run over a differently shuffled file still yields identical labels.

The diagnosis column is the network’s target, and the remaining 30 columns (10 measurements $\times$ the `mean/se/extreme` statistics) form the feature matrix.

4.2 Standardization (z-score)

Features are standardized to zero mean and unit variance per feature:

$$x'_j = \frac{x_j - \mu_j}{\sigma_j}$$

The statistics (μ_j, σ_j) are computed *only on the training split* and then applied unchanged to the validation and test sets — fitting them on the test set would leak information into the training pipeline and make the reported generalization optimistic. This is `standardize_from_train` in `src/app/training.rs`, built on `mean_axis(0)` and `std_axis(0, 0.0)`; standard deviations are floored to 10^{-12} so a constant column cannot divide by zero.

The same idea is also packaged as reusable scalers — `StandardScaler` and `MinMaxScaler` behind a `Normalizer` trait with `fit / transform / fit_transform (src/data/preprocessing.rs)`. The training path uses the inline `standardize_from_train`; the generic classes exist for the same purpose and are covered by unit tests against `data/data.csv`.

4.3 Train/Validation Split

The evaluation grid asks whether the dataset is split in two. The project offers two mechanisms:

1. **Automatic holdout during training.** `train_from_dataset (src/app/training.rs)` uses the first 85% of the rows for training and the remaining 15% for validation, so every run produces a real train/validation loss gap, which the callbacks of Section 7 monitor.
2. **split subcommand.** `run_split (src/app/split.rs)` writes two CSV files (`--train-out`, `--val-out`) for a configurable ratio $r \in (0, 1)$, mirroring the subject’s `data_training.csv{}` / `data_test.csv{}` workflow. Underneath, `train_test_split (src/data/split.rs)` performs a seeded shuffle (a fixed seed reproduces the split exactly) and optionally *stratifies*: rows are grouped by a feature column (or by the label), and each group is split separately so the class balance is preserved in both halves.

4.4 Model Persistence and Prediction

After training, `Network::save (src/network/persist.rs)` writes a single JSON file holding the complete learned state: the topology (per layer: weight matrix, bias vector, activation) together with the standardization statistics `feature_mean` and `feature_std` — answering the grid’s “is the topology and the weights saved?”:

```
{
  "learning_rate": 0.0005,
  "feature_mean": [...30 values...],
  "feature_std": [...30 values...],
  "layers": [
```



```

    { "weights": [...30x24...], "bias": [...], "activation": "sigmoid" },
    { "weights": [...24x12...], "bias": [...], "activation": "sigmoid" },
    { "weights": [...12x2...], "bias": [...], "activation": "softmax" }
  ]
}

```

`Network::load` validates the file (at least three layers, matching weight/bias dimensions) and restores the network. The prediction program `run_predict` (`src/app/predict.rs`) then:

1. loads the model,
2. standardizes the test features with the *saved* training statistics (falling back to fitting on the prediction file if the model predates scaler storage),
3. runs a forward pass (`network.predict`) and maps each row to a class — argmax over the softmax outputs, or a 0.5 threshold for a single sigmoid output,
4. reports binary cross-entropy and accuracy on the test set, i.e. the subject's mandatory final evaluation.

This closes the loop: train on `data_training.csv`, save the model, then score `data_test.csv` with the saved weights and the same standardization.

5 Backpropagation

5.1 Chain Rule — Scalar Refresher

Backpropagation is the chain rule of calculus applied to a neural network's computation graph. For a composition of functions $\mathcal{L} = f(g(h(\mathbf{x})))$, the derivative with respect to the input is:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial f}{\partial g} \frac{\partial g}{\partial h} \frac{\partial h}{\partial \mathbf{x}}$$

Each factor is a local Jacobian. The key insight of backpropagation is that we compute these factors from right to left (output $\rightarrow$ input), reusing intermediate results instead of recomputing them. This is critical: the naive forward-mode approach would be exponential in the number of layers.

In a network of L layers, the computation graph is a chain:

$$\mathbf{x} = \mathbf{a}^{(0)} \rightarrow \mathbf{z}^{(1)} \rightarrow \mathbf{a}^{(1)} \rightarrow \mathbf{z}^{(2)} \rightarrow \mathbf{a}^{(2)} \rightarrow \dots \rightarrow \mathbf{z}^{(L)} \rightarrow \mathbf{a}^{(L)} = \hat{\mathbf{y}} \rightarrow \mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$$

Each layer ℓ performs:

$$\mathbf{z}^{(\ell)} = \mathbf{a}^{(\ell-1)} \mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)} \quad \mathbf{a}^{(\ell)} = \phi^{(\ell)}(\mathbf{z}^{(\ell)})$$

5.2 The Three Core Gradients of One Layer

Consider a single layer ℓ . We want three quantities:

- (i) $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(\ell)}}$ — how much each weight contributed to the loss
- (ii) $\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(\ell)}}$ — how much each bias contributed to the loss
- (iii) $\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(\ell-1)}}$ — gradient w.r.t. the *input* of this layer,
to be propagated further backward

All three are computed from the *upstream gradient* $\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(\ell)}}$ that arrives from the layer above (closer to the output). The upstream gradient tells us: “given how the loss changes when the output $\mathbf{a}^{(\ell)}$ changes, how should we adjust each parameter?”

Step 0: Gate through the activation function

The activation function sits between $\mathbf{z}^{(\ell)}$ (pre-activation) and $\mathbf{a}^{(\ell)}$ (post-activation). By the chain rule:

$$\boxed{\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(\ell)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(\ell)}} \odot \phi'(\mathbf{z}^{(\ell)})} \quad (1)$$

where $\odot$ is the element-wise (Hadamard) product. This is because the activation function applies independently to each element of $\mathbf{z}$: $a_j = \phi(z_j)$, so $\partial a_j / \partial z_j = \phi'(z_j)$ and $\partial a_j / \partial z_k = 0$ for $j \neq k$.

We denote this gated gradient as $\Delta^{(\ell)}$:

$$\Delta^{(\ell)} \triangleq \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(\ell)}}$$

This is the central quantity from which all three parameter gradients derive.

Step 1: Gradient w.r.t. the bias $\mathbf{b}^{(\ell)}$

The bias enters the forward computation as:

$$z_j^{(\ell)} = \sum_i a_i^{(\ell-1)} W_{ij}^{(\ell)} + b_j^{(\ell)}$$

Only the j -th bias element affects $z_j^{(\ell)}$. Therefore:

$$\frac{\partial z_j^{(\ell)}}{\partial b_j^{(\ell)}} = 1$$

Applying the chain rule through the full path $\mathcal{L} \rightarrow \mathbf{a}^{(\ell)} \rightarrow \mathbf{z}^{(\ell)} \rightarrow \mathbf{b}^{(\ell)}$:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial b_j^{(\ell)}} &= \sum_k \underbrace{\frac{\partial \mathcal{L}}{\partial a_k^{(\ell)}}}_{\text{upstream}} \cdot \underbrace{\frac{\partial a_k^{(\ell)}}{\partial z_k^{(\ell)}}}_{\text{activation } \mathbf{a}^{(\ell)}} \cdot \underbrace{\frac{\partial z_k^{(\ell)}}{\partial b_j^{(\ell)}}}_{\delta_{kj}} \\ &= \sum_k \frac{\partial \mathcal{L}}{\partial a_k^{(\ell)}} \frac{\partial a_k^{(\ell)}}{\partial z_k^{(\ell)}} \delta_{kj} \\ &= \frac{\partial \mathcal{L}}{\partial a_j^{(\ell)}} \frac{\partial a_j^{(\ell)}}{\partial z_j^{(\ell)}} \cdot 1 \\ &= \Delta_j^{(\ell)} \end{aligned}$$

where δ_{kj} is the Kronecker delta (1 if $k = j$, 0 otherwise). The final line uses the definition of the gated gradient from Step 0: $\Delta_j^{(\ell)} = \frac{\partial \mathcal{L}}{\partial a_j^{(\ell)}} \frac{\partial a_j^{(\ell)}}{\partial z_j^{(\ell)}}$.

Or in vector form:

$$\boxed{\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(\ell)}} = \Delta^{(\ell)}} \quad (2)$$

This is the gradient for a *single sample*. For a batch of B samples, each sample produces its own $\Delta_i^{(\ell)}$, and the total gradient is the sum over the batch (or mean, depending on the loss function's normalization convention).

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(\ell)}} = \sum_{i=1}^B \Delta_i^{(\ell)} \quad (2b)$$

Step 2: Gradient w.r.t. the weights $\mathbf{W}^{(\ell)}$

From $z_j^{(\ell)} = \sum_i a_i^{(\ell-1)} W_{ij}^{(\ell)} + b_j^{(\ell)}$:

$$\frac{\partial z_j^{(\ell)}}{\partial W_{ij}^{(\ell)}} = a_i^{(\ell-1)}$$

The chain rule through the full path $\mathcal{L} \rightarrow \mathbf{a}^{(\ell)} \rightarrow \mathbf{z}^{(\ell)} \rightarrow \mathbf{W}^{(\ell)}$:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial W_{ij}^{(\ell)}} &= \sum_k \underbrace{\frac{\partial \mathcal{L}}{\partial a_k^{(\ell)}}}_{\text{upstream}} \cdot \underbrace{\frac{\partial a_k^{(\ell)}}{\partial z_k^{(\ell)}}}_{\text{activation } \mathbf{a}^{(\ell)}} \cdot \underbrace{\frac{\partial z_k^{(\ell)}}{\partial W_{ij}^{(\ell)}}}_{a_i^{(\ell-1)} \delta_{kj}} \\ &= \sum_k \frac{\partial \mathcal{L}}{\partial a_k^{(\ell)}} \frac{\partial a_k^{(\ell)}}{\partial z_k^{(\ell)}} \delta_{kj} a_i^{(\ell-1)} \\ &= \frac{\partial \mathcal{L}}{\partial a_j^{(\ell)}} \frac{\partial a_j^{(\ell)}}{\partial z_j^{(\ell)}} a_i^{(\ell-1)} \\ &= \Delta_j^{(\ell)} a_i^{(\ell-1)} \end{aligned}$$

In matrix form, this outer product is:

$$\boxed{\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(\ell)}} = (\mathbf{a}^{(\ell-1)})^T \Delta^{(\ell)}} \quad (3)$$

Let us verify the dimensions:

$$\underbrace{(\mathbf{a}^{(\ell-1)})^T}_{d_{\ell-1} \times 1} \underbrace{\Delta^{(\ell)}}_{1 \times d_\ell} = \underbrace{\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(\ell)}}}_{d_{\ell-1} \times d_\ell}$$

For a batch of B samples, the matrix $\mathbf{A}^{(\ell-1)}$ has shape $B \times d_{\ell-1}$ and $\Delta^{(\ell)}$ has shape $B \times d_\ell$. The batch gradient becomes:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(\ell)}} = (\mathbf{A}^{(\ell-1)})^T \Delta^{(\ell)} \quad \underbrace{(d_{\ell-1} \times B) (B \times d_\ell)}_{\rightarrow d_{\ell-1} \times d_\ell} \quad (3b)$$

The matrix multiplication implicitly sums over the batch dimension.

Step 3: Gradient w.r.t. the layer input $\mathbf{a}^{(\ell-1)}$

This is the gradient that *backpropagates* — it is passed to the previous (earlier) layer as its upstream gradient. From $z_j^{(\ell)} = \sum_i a_i^{(\ell-1)} W_{ij}^{(\ell)} + b_j^{(\ell)}$:

$$\frac{\partial z_j^{(\ell)}}{\partial a_i^{(\ell-1)}} = W_{ij}^{(\ell)}$$

The chain rule through the full path $\mathcal{L} \rightarrow \mathbf{a}^{(\ell)} \rightarrow \mathbf{z}^{(\ell)} \rightarrow \mathbf{a}^{(\ell-1)}$:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial a_i^{(\ell-1)}} &= \sum_j \frac{\partial \mathcal{L}}{\partial a_j^{(\ell)}} \frac{\partial a_j^{(\ell)}}{\partial z_j^{(\ell)}} \frac{\partial z_j^{(\ell)}}{\partial a_i^{(\ell-1)}} \\ &= \sum_j \frac{\partial \mathcal{L}}{\partial a_j^{(\ell)}} \frac{\partial a_j^{(\ell)}}{\partial z_j^{(\ell)}} W_{ij}^{(\ell)} \\ &= \sum_j \Delta_j^{(\ell)} W_{ij}^{(\ell)} \end{aligned}$$

In matrix form, this is a post-multiplication by the transposed weight matrix:

$$\boxed{\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(\ell-1)}} = \mathbf{\Delta}^{(\ell)} (\mathbf{W}^{(\ell)})^T} \quad (4)$$

Dimensions:

$$\underbrace{\mathbf{\Delta}^{(\ell)}}_{1 \times d_\ell} \underbrace{(\mathbf{W}^{(\ell)})^T}_{d_\ell \times d_{\ell-1}} = \underbrace{\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(\ell-1)}}}_{1 \times d_{\ell-1}}$$

For a batch:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{A}^{(\ell-1)}} = \mathbf{\Delta}^{(\ell)} (\mathbf{W}^{(\ell)})^T \underbrace{(B \times d_\ell) (d_\ell \times d_{\ell-1})}_{\rightarrow B \times d_{\ell-1}} \quad (4b)$$

This is then fed as the upstream gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(\ell)}}$ to layer $\ell - 1$.

5.3 Summary of One Layer

Given the upstream gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(\ell)}}$ and cached values $\mathbf{z}^{(\ell)}$ (pre-activation) and $\mathbf{a}^{(\ell-1)}$ (layer input):

$$\begin{aligned} \text{(gate)} \quad \mathbf{\Delta}^{(\ell)} &= \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(\ell)}} \odot \phi'(\mathbf{z}^{(\ell)}) \quad (\text{shape } B \times d_\ell) \\ \text{(bias grad)} \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(\ell)}} &= \sum_{i=1}^B \mathbf{\Delta}_i^{(\ell)} \quad (\text{shape } d_\ell) \\ \text{(weight grad)} \quad \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(\ell)}} &= (\mathbf{A}^{(\ell-1)})^T \mathbf{\Delta}^{(\ell)} \quad (\text{shape } d_{\ell-1} \times d_\ell) \\ \text{(backprop)} \quad \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(\ell-1)}} &= \mathbf{\Delta}^{(\ell)} (\mathbf{W}^{(\ell)})^T \quad (\text{shape } B \times d_{\ell-1}) \end{aligned}$$

5.4 Full Network Algorithm

1. **Forward pass:** For $\ell = 1, \dots, L$:

$$\begin{aligned} \mathbf{Z}^{(\ell)} &= \mathbf{A}^{(\ell-1)} \mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)} \\ \mathbf{A}^{(\ell)} &= \phi^{(\ell)}(\mathbf{Z}^{(\ell)}) \end{aligned}$$

Cache $\mathbf{A}^{(\ell-1)}$ and $\mathbf{Z}^{(\ell)}$ on each layer.

2. **Loss gradient:**

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(L)}} = \frac{\partial \mathcal{L}}{\partial \hat{\mathbf{y}}}$$

computed from the chosen loss function.

3. **Backward pass:** For $\ell = L, L-1, \dots, 1$:

$$\begin{aligned} \Delta^{(\ell)} &= \underbrace{\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(\ell)}}}_{\text{upstream gradient}} \odot \phi'(\underbrace{\mathbf{z}^{(\ell)}}_{\text{weighted sum}}) \\ \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(\ell)}} &= (\mathbf{A}^{(\ell-1)})^T \Delta^{(\ell)} \\ \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(\ell)}} &= \sum_{i=1}^B \Delta_i^{(\ell)} \\ \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(\ell-1)}} &= \Delta^{(\ell)} (\mathbf{W}^{(\ell)})^T \end{aligned}$$

Store $(\partial \mathcal{L} / \partial \mathbf{W}^{(\ell)}, \partial \mathcal{L} / \partial \mathbf{b}^{(\ell)})$ and feed $\partial \mathcal{L} / \partial \mathbf{a}^{(\ell-1)}$ to the next (earlier) iteration.

5.5 Concrete Numerical Example

Consider a 3-layer network with sigmoid hidden activations and a single sigmoid output, processing one sample:

$$d_0 = 2 \quad d_1 = 3 \quad d_2 = 2 \quad d_3 = 1$$

Forward pass

Let the input be $\mathbf{x} = [0.5, 1.0]$, target $y = 1.0$.

Layer 1 (input $\rightarrow$ 3 neurons):

$$\mathbf{W}^{(1)} = \begin{bmatrix} 0.1 & 0.4 & 0.7 \\ 0.2 & 0.5 & 0.8 \end{bmatrix} \quad \mathbf{b}^{(1)} = [0.0 \quad 0.0 \quad 0.0]$$

$$\mathbf{z}^{(1)} = [0.5 \quad 1.0] \begin{bmatrix} 0.1 & 0.4 & 0.7 \\ 0.2 & 0.5 & 0.8 \end{bmatrix} = [0.25 \quad 0.70 \quad 1.15]$$

$$\mathbf{a}^{(1)} = \sigma(\mathbf{z}^{(1)}) = [0.562 \quad 0.668 \quad 0.760]$$

Layer 2 (3 $\rightarrow$ 2 neurons):

$$\mathbf{W}^{(2)} = \begin{bmatrix} 0.2 & 0.6 \\ 0.3 & 0.7 \\ 0.4 & 0.8 \end{bmatrix} \quad \mathbf{b}^{(2)} = [0.0 \quad 0.0]$$

$$\mathbf{z}^{(2)} = [0.562 \quad 0.668 \quad 0.760] \begin{bmatrix} 0.2 & 0.6 \\ 0.3 & 0.7 \\ 0.4 & 0.8 \end{bmatrix} = [0.620 \quad 1.362]$$

$$\mathbf{a}^{(2)} = \sigma(\mathbf{z}^{(2)}) = [0.650 \quad 0.796]$$

Layer 3 (2 → 1 output):

$$\mathbf{W}^{(3)} = \begin{bmatrix} 0.5 \\ 0.9 \end{bmatrix} \quad \mathbf{b}^{(3)} = [0.0]$$

$$\mathbf{z}^{(3)} = [0.650 \ 0.796] \begin{bmatrix} 0.5 \\ 0.9 \end{bmatrix} = 1.041$$

$$\mathbf{a}^{(3)} = \sigma(1.041) = 0.739$$

Output: $\hat{y} = 0.739$, target: $y = 1.0$.

Loss gradient (BCE, single sample)

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})} = \frac{0.739 - 1.0}{0.739 \times 0.261} = \frac{-0.261}{0.193} = -1.352$$

Backward pass

Layer 3 (output, $\ell = 3$):

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(3)}} = [-1.352] \quad \mathbf{z}^{(3)} = [1.041] \quad \sigma'(1.041) = 0.739 \times (1 - 0.739) = 0.193$$

$$\Delta^{(3)} = [-1.352] \odot [0.193] = [-0.261]$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(3)}} = \begin{bmatrix} 0.650 \\ 0.796 \end{bmatrix} [-0.261] = \begin{bmatrix} -0.170 \\ -0.208 \end{bmatrix}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(3)}} = [-0.261]$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(2)}} = [-0.261] [0.5 \ 0.9] = [-0.131 \ -0.235]$$

The gradient $\partial \mathcal{L} / \partial \mathbf{a}^{(2)} = [-0.131 \ -0.235]$ is now the upstream gradient for layer 2.

Layer 2 ($\ell = 2$):

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(2)}} = [-0.131 \ -0.235] \quad \mathbf{z}^{(2)} = [0.620 \ 1.362]$$

$$\sigma'(0.620) = 0.650 \times 0.350 = 0.228$$

$$\sigma'(1.362) = 0.796 \times 0.204 = 0.162$$

$$\phi'(\mathbf{z}^{(2)}) = [0.228 \ 0.162]$$

$$\Delta^{(2)} = [-0.131 \ -0.235] \odot [0.228 \ 0.162] = [-0.030 \ -0.038]$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(2)}} = \begin{bmatrix} 0.562 \\ 0.668 \\ 0.760 \end{bmatrix} [-0.030 \ -0.038] = \begin{bmatrix} -0.017 & -0.021 \\ -0.020 & -0.025 \\ -0.023 & -0.029 \end{bmatrix}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(2)}} = [-0.030 \ -0.038]$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(1)}} = [-0.030 \ -0.038] \begin{bmatrix} 0.2 & 0.3 & 0.4 \\ 0.6 & 0.7 & 0.8 \end{bmatrix} = [-0.029 \ -0.036 \ -0.042]$$

Layer 1 ($\ell = 1$):

$$\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(1)}} = [-0.029 \quad -0.036 \quad -0.042] \quad \mathbf{z}^{(1)} = [0.25 \quad 0.70 \quad 1.15]$$

$$\sigma'(0.25) = 0.562 \times 0.438 = 0.246$$

$$\sigma'(0.70) = 0.668 \times 0.332 = 0.222$$

$$\sigma'(1.15) = 0.760 \times 0.240 = 0.182$$

$$\phi'(\mathbf{z}^{(1)}) = [0.246 \quad 0.222 \quad 0.182]$$

$$\Delta^{(1)} = [-0.029 \quad -0.036 \quad -0.042] \odot [0.246 \quad 0.222 \quad 0.182] = [-0.007 \quad -0.008 \quad -0.008]$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \begin{bmatrix} 0.5 \\ 1.0 \end{bmatrix} [-0.007 \quad -0.008 \quad -0.008] = \begin{bmatrix} -0.004 & -0.004 & -0.004 \\ -0.007 & -0.008 & -0.008 \end{bmatrix}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(1)}} = [-0.007 \quad -0.008 \quad -0.008]$$

The gradient w.r.t. the network input $\partial \mathcal{L} / \partial \mathbf{x}$ is discarded (no earlier parameters depend on it).

Parameter update (SGD with $\eta = 0.1$)

$$\mathbf{W}^{(3)} \leftarrow \begin{bmatrix} 0.5 \\ 0.9 \end{bmatrix} - 0.1 \times \begin{bmatrix} -0.170 \\ -0.208 \end{bmatrix} = \begin{bmatrix} 0.517 \\ 0.921 \end{bmatrix}$$

$$\mathbf{W}^{(2)} \leftarrow \begin{bmatrix} 0.2 & 0.6 \\ 0.3 & 0.7 \\ 0.4 & 0.8 \end{bmatrix} - 0.1 \times \begin{bmatrix} -0.017 & -0.021 \\ -0.020 & -0.025 \\ -0.023 & -0.029 \end{bmatrix} = \begin{bmatrix} 0.202 & 0.602 \\ 0.302 & 0.702 \\ 0.402 & 0.803 \end{bmatrix}$$

$$\mathbf{W}^{(1)} \leftarrow \begin{bmatrix} 0.1 & 0.4 & 0.7 \\ 0.2 & 0.5 & 0.8 \end{bmatrix} - 0.1 \times \begin{bmatrix} -0.004 & -0.004 & -0.004 \\ -0.007 & -0.008 & -0.008 \end{bmatrix} = \begin{bmatrix} 0.100 & 0.400 & 0.700 \\ 0.201 & 0.501 & 0.801 \end{bmatrix}$$

All biases are adjusted similarly. Note that the gradients are largest near the output (layer 3 sees $\Delta \approx -0.26$ while layer 1 sees $\Delta \approx -0.008$) — this is the *vanishing gradient* effect with sigmoid activations.

5.6 Correlation to Rust Code

The implementation mirrors the mathematics. The following table maps each mathematical quantity to its Rust representation:

Math	Code variable	Shape	Rust expression
$\mathbf{A}^{(\ell-1)}$	<code>input_cache</code>	$(B, d_{\ell-1})$	<code>self.input_cache.as_ref().unwrap()</code>
$\mathbf{Z}^{(\ell)}$	<code>weighted_sum_cache</code>	(B, d_{ℓ})	<code>self.weighted_sum_cache.as_ref().unwrap()</code>
$\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(\ell)}}$	<code>upstream</code>	(B, d_{ℓ})	<code>upstream</code>
$\Delta^{(\ell)}$	<code>delta</code> (matches Δ)	(B, d_{ℓ})	<code>self.activation.backward(weighted_sum, upstream) (grad $\odot$ $\phi'(\mathbf{x})$)</code>
$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(\ell)}}$	<code>grad_weights</code>	$(d_{\ell-1}, d_{\ell})$	<code>input.t().dot(&delta)</code>
$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(\ell)}}$	<code>grad_bias</code>	$(d_{\ell},)$	<code>delta.sum_axis(Axis(0))</code>
$\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(\ell-1)}}$	<code>grad_input</code>	$(B, d_{\ell-1})$	<code>delta.dot(&self.weights.t())</code>

The implementation:

```

fn backward(&self, upstream: &Array2<f64>)
-> (Array2<f64>, Array2<f64>, Array1<f64>)
{
    let weighted_sum = self.weighted_sum_cache.as_ref().unwrap(); // z: weighted sum (pre-activation)
    let input = self.input_cache.as_ref().unwrap();                // a_prev: previous layer output

    let delta = self.activation.backward(weighted_sum, upstream); // upstream gradient * f'(z)
    let grad_input = delta.dot(&self.weights.t());
    let grad_weights = input.t().dot(&delta);
    let grad_bias = delta.sum_axis(Axis(0));

    (grad_input, grad_weights, grad_bias)
}

```

5.7 Intuition in Natural Language

1. **Forward:** Each layer takes the output of the previous layer, multiplies by its weights, adds bias, and applies a nonlinearity. The intermediate values are cached.
2. **Loss:** The final output is compared to the target via a loss function, producing a scalar $\mathcal{L}$.
3. **Backward:** For each layer, in reverse order:
 - “How much did each pre-activation z_j affect the loss?” Answer: $\Delta_j = \partial\mathcal{L}/\partial a_j \times \phi'(z_j)$. If the activation was saturated ($\phi' \approx 0$), the gradient is killed.
 - “How much did each weight W_{ij} contribute?” Answer: the product of the incoming activation a_i (which carries the signal forward) and Δ_j (which carries the error backward).
 - “How much did the bias b_j contribute?” Answer: directly Δ_j .
 - “What error signal should we send to the previous layer?” Answer: Δ times the transpose of the weight matrix, distributing responsibility backward.
4. **Update:** Each parameter is adjusted opposite to its gradient: $\theta \leftarrow \theta - \eta \cdot \partial\mathcal{L}/\partial\theta$.

The process is a direct application of the chain rule, with the caching of $\mathbf{A}^{(\ell-1)}$ and $\mathbf{Z}^{(\ell)}$ making the backward pass computationally efficient.

6 Gradient Descent

Backpropagation (Section 5) produces the gradients of the loss with respect to the weights and biases of every layer, $\partial\mathcal{L}/\partial\mathbf{W}^{(\ell)}$ and $\partial\mathcal{L}/\partial\mathbf{b}^{(\ell)}$. Every optimizer in this project applies the same skeleton,

$$\theta_{t+1} = \theta_t - \eta \cdot \Delta_t$$

where Δ_t is the weight/bias gradient vector from backpropagation (one component per weight and bias) or a transformation of it, and η is the learning rate — constant per run, configurable via YAML (default 0.0314). The variants implemented in `src/training/optimizer.rs` are selected with the `optimizer` YAML field (`sgd` | `adam`), default `sgd`.

Per the project subject, plain **gradient descent** (SGD) is mandatory: the training program must learn with backpropagation and gradient descent. **Adam** is optional and belongs to the bonus part, under “a more complex optimization function”.

6.1 Stochastic Gradient Descent (SGD) (mandatory)

The trainer (`src/training/trainer.rs`) shuffles the samples each epoch and splits them into groups of `batch_size` (default 8; `src/training/batch.rs`). The shuffle happens once at the start of each epoch, so the groups are a random partition of the N samples — every sample is used exactly once per epoch (sampling without replacement). Each group is therefore a uniformly random subset, and its mean gradient is an unbiased estimate of the full-data gradient. One optimizer update is applied per group, using the gradient averaged over the group:

```
let pred      = network.forward(&x_batch); // B samples at once
let batch_size = pred.nrows() as f64;    // B
let mut upstream = self.loss_fn.gradient(pred.view(), y_batch.view());
upstream /= batch_size; // mean-gradient normalization
let gradients = network.backward(upstream); // dL/dW, dL/db (Section 5)
self.optimizer.update(network, &gradients); // update: theta -= lr * g
```

The loss gradient is computed per sample w.r.t. the predictions and averaged over the batch ($\div B$); backpropagation (Section 5) then chains it into the weight and bias gradients $\mathbf{g} = (\partial\mathcal{L}/\partial\mathbf{W}^{(\ell)}, \partial\mathcal{L}/\partial\mathbf{b}^{(\ell)})_{\ell=1}^L$, and each parameter moves opposite to its gradient:

$$\mathbf{W}^{(\ell)} \leftarrow \mathbf{W}^{(\ell)} - \eta \frac{\partial\mathcal{L}}{\partial\mathbf{W}^{(\ell)}} \quad \mathbf{b}^{(\ell)} \leftarrow \mathbf{b}^{(\ell)} - \eta \frac{\partial\mathcal{L}}{\partial\mathbf{b}^{(\ell)}}$$

With `batch_size = 1` this is stochastic SGD, with $1 < B < N$ mini-batch SGD, and with $B = N$ full-batch GD. In code:

```
layer.weights = &layer.weights - (&(grad.weights.clone()) * self.learning_rate);
layer.bias    = &layer.bias    - (&(grad.bias.clone()) * self.learning_rate);
```

6.2 Adam (Adaptive Moment Estimation) (bonus)

Combines momentum (first moment $\mathbf{m}$) with per-parameter scaling (second moment $\mathbf{v}$), both bias-corrected:

$$\begin{aligned} \mathbf{m}_{t+1} &= \beta_1 \mathbf{m}_t + (1 - \beta_1) \mathbf{g}_t \\ \mathbf{v}_{t+1} &= \beta_2 \mathbf{v}_t + (1 - \beta_2) \mathbf{g}_t^2 \\ \hat{\mathbf{m}}_{t+1} &= \frac{\mathbf{m}_{t+1}}{1 - \beta_1^{t+1}}, \quad \hat{\mathbf{v}}_{t+1} = \frac{\mathbf{v}_{t+1}}{1 - \beta_2^{t+1}} \\ \theta_{t+1} &= (1 - \eta\lambda) \theta_t - \frac{\eta \hat{\mathbf{m}}_{t+1}}{\sqrt{\hat{\mathbf{v}}_{t+1}} + \varepsilon} \end{aligned}$$

Defaults $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$. The bias correction compensates for $\mathbf{m}$ and $\mathbf{v}$ starting at zero; without it the first steps would be too small (first moment) or too large (second moment). The update also applies *decoupled* L2 weight decay (AdamW-style, the `weight_decay` YAML field, default 0): the factor $(1 - \eta\lambda)$ shrinks the weights before the Adam correction, so the decay does not enter the first/second moments. The tuned model uses $\lambda = 0.5$. In code (`src/training/optimizer.rs`):

```
let decay = 1.0 - self.learning_rate * self.weight_decay;
let bc1   = 1.0 - self.beta1.powi(self.t as i32);
let bc2   = 1.0 - self.beta2.powi(self.t as i32);
let mw_hat = mw.mapv(|v| v / bc1);
let mb_hat = mb.mapv(|v| v / bc1);
let vw_hat = vw.mapv(|v| v / bc2);
let vb_hat = vb.mapv(|v| v / bc2);
layer.weights = &layer.weights * decay
```

```

- &(mw_hat * self.learning_rate / vw_hat.mapv(|v| v.sqrt() + self.epsilon));
layer.bias    = &layer.bias
- &(mb_hat * self.learning_rate / vb_hat.mapv(|v| v.sqrt() + self.epsilon));

```

Note that only the weights are shrunk; the biases are updated without decay.

6.3 Weight Initialization

Implemented in `src/network/initializer.rs` and selected per layer with the `initializer` YAML field (default `he`):

Initializer	Code	Scale
<code>he</code>	<code>Uniform::new(-L, L), $L = (2.0/rows).sqrt()$</code>	$\sqrt{2/n_{in}}$
<code>xavier</code>	<code>Uniform::new(-L, L), $L = (6.0/(rows + cols)).sqrt()$</code>	$\sqrt{6/(n_{in} + n_{out})}$
<code>random</code>	<code>Array2::random(_, Standard)</code>	$\mathcal{U}(0, 1)$

where `rows` = n_{in} and `cols` = n_{out} . Biases are initialized to zero (`src/network/layer.rs`).

6.4 Overall Training Phase (mandatory)

The optimizers above are driven by the trainer (`src/training/trainer.rs`). The complete loop for one run:

1. **Setup** (`src/app/training.rs`): build the network and trainer from the YAML config (defaults `epochs=84`, `batch_size=8`, `learning_rate=0.0314`, `optimizer=sgd`); the CLI trains with `LossFunction::CategoricalCrossEntropy`.
2. **Per epoch**:
 - (a) Shuffle the training indices.
 - (b) Walk the shuffled data in groups of `batch_size` (`create_batches`); per group: forward over the whole group $\rightarrow$ per-sample loss gradient w.r.t. predictions (Section 3) $\rightarrow$ normalize by `pred.nrows()` $\rightarrow$ backpropagation into weight/bias gradients (Section 5) $\rightarrow$ one optimizer update.
 - (c) Aggregate `train_loss` as the mean batch loss and recompute train accuracy and precision from a full forward pass; repeat on the validation set if provided.
 - (d) Notify callbacks (`on_epoch_end`); stop early if a callback requests it (`should_stop()`).
3. **Report** the final train and validation metrics.

Epoch-level pseudocode, abridged from the implementation:

```

for epoch in 0..epochs {
  indices.shuffle(&mut rng);
  create_batches(x_train, y_train, &indices, batch_size)
    .sum_by(|x_batch, y_batch| {
      let pred          = network.forward(&x_batch);
      let batch_loss    = loss_fn.compute(pred.view(), y_batch.view()).mean();
      let mut upstream  = loss_fn.gradient(pred.view(), y_batch.view());
      upstream          /= pred.nrows() as f64;
      let gradients     = network.backward(upstream);
      optimizer.update(network, &gradients);
      batch_loss
    });
}

```

The callbacks attached by the CLI are the live GUI monitor, the history recorder, the early-stopping callback (monitored metric, mode min/max, patience, min-delta), and the progress logger. The first three implement the subject’s bonus features of displaying multiple learning curves on one graph (live monitor), keeping an historic of the metric (history recorder), and early stopping. Together with the Adam optimizer of this section and the precision metric of Section 8, these are exactly the five bonuses listed in the subject.

7 Overfitting

7.1 Definition

Overfitting occurs when a model learns the training data too well, capturing noise and random fluctuations instead of the underlying pattern. The model performs well on training data but poorly on unseen data.

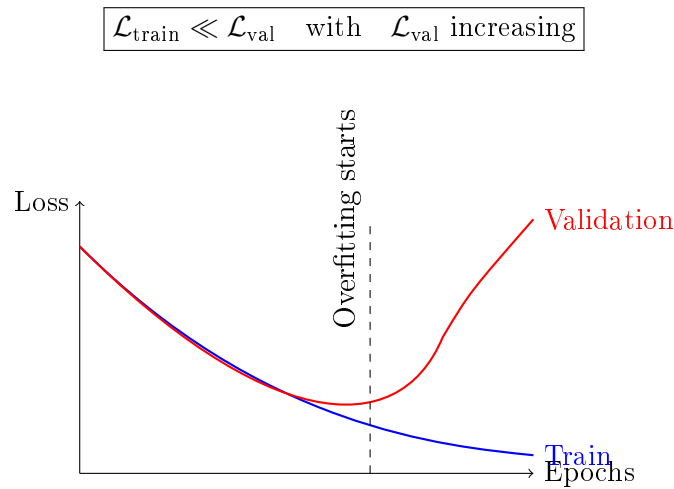


Figure 1: Overfitting signature: val loss decreasing then diverging upward.

7.2 Why It Happens

- **Model too complex:** Too many parameters relative to training samples.
- **Training too long:** Without early stopping, the model eventually memorizes noise.
- **Insufficient data:** The training set doesn’t represent the full data distribution.
- **No regularization:** The model has no constraints to limit complexity.

7.3 Early Stopping — available in this project

Monitor a validation metric during training; stop when it stops improving for p epochs (patience):

$$\text{Stop if } \mathcal{L}_{\text{val}}^{(t)} > \mathcal{L}_{\text{val}}^{(\text{best})} \text{ for } p \text{ consecutive epochs}$$

This prevents the model from entering the divergence phase in Figure 1. In this project, early stopping tracks a configurable metric (loss, accuracy, etc.) in either min or max mode. It is disabled by default; the best-epoch weights are tracked regardless and restored after training, so the saved model always corresponds to the best epoch rather than the last one.

7.4 Detection in Practice

To detect overfitting during training:

1. **Compare train vs. val metrics:** A gap larger than a few percent is suspicious.
2. **Plot learning curves:** Look for the characteristic divergence pattern (Fig. 1).
3. **Inspect weight magnitudes:** Unusually large weights suggest the model is fitting noise.
4. **Test set:** The final evaluation on a completely unseen test set gives the true generalization error.

7.5 This Project’s Architecture

The current model (2 hidden layers of 24 and 12 neurons, Adam, learning rate $5 \cdot 10^{-4}$, weight decay 0.5, batch size 16, 250 epochs) shows:

$$\begin{aligned}\text{Train loss} &= 0.057 & \text{Val loss} &= 0.065 \\ \text{Train accuracy} &= 98.8\% & \text{Val accuracy} &= 98.8\%\end{aligned}$$

The negligible train/val gap indicates no significant overfitting. The architecture (~ 1070 parameters) is appropriate for the 483 training samples. The weight decay was tuned by evaluating on random 80/20 shuffled splits (see 8); stronger decay softens the model on a few ambiguous samples but further decay underfits. If overfitting appeared, earlier early stopping or a smaller network would be the first mitigations.

8 Evaluation Metrics

Beyond the loss, the model is scored during and after training with classification metrics. The code implements these in `compute_classification_scores_from_labels` and `accuracy` (`src/training/metrics.rs`).

Per the project subject, **accuracy** (together with the loss) is part of the mandatory phase: the two learning curves of loss and accuracy must be displayed at the end of training. **Precision** is optional and belongs to the bonus part, under “evaluate the learning phase with multiple metrics”.

8.1 Turning predictions into labels

A prediction row is decoded to a class label depending on the output width:

1. **Single output column** (binary classification): the predicted class is 1 if $p \geq 0.5$, otherwise 0.
2. **Multiple output columns** (multiclass): the predicted class is the index of the largest logit, i.e. $\hat{c} = \arg \max_k p_k$.

Targets are given as class indices $c_i \in \{0, \dots, K-1\}$.

8.2 Accuracy (mandatory)

The fraction of correct predictions:

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{c}_i = c_i]$$

8.3 Precision with macro-averaging (bonus)

Both `accuracy` and `precision` are produced by the single function `compute_classification_scores_from_labels` in `src/training/metrics.rs`, which decodes every prediction row into a class label as described above, then works as follows:

1. **Class set.** The set of classes $\mathcal{C} = \{\hat{c}_i\} \cup \{c_i\}$ is the union of the predicted and true labels actually seen in the batch (a `BTreeSet` in the code). This is the set over which the macro average is taken.
2. **Per-class true and false positives.** For each class $k \in \mathcal{C}$ the code counts:

$$TP_k = \#\{i : \hat{c}_i = k \text{ and } c_i = k\}$$
$$FP_k = \#\{i : \hat{c}_i = k \text{ and } c_i \neq k\}$$

3. **Per-class precision.**

$$\text{Precision}_k = \frac{TP_k}{TP_k + FP_k}$$

set to 0 when the denominator $TP_k + FP_k$ is zero (no prediction of class k).

4. **Macro-average.** The per-class scores are summed and divided by the number of classes $|\mathcal{C}|$:

$$\text{Precision} = \frac{1}{|\mathcal{C}|} \sum_{k \in \mathcal{C}} \text{Precision}_k$$

so every class receives the same weight regardless of its frequency.

Accuracy is the fraction of correct predictions (the first of the two scores returned by the same function). The scores are aggregated into a `Metrics` struct (with `train_/_val_` variants) reported at the end of every epoch.

8.4 Performance testing note

The hyperparameters of the submitted model were chosen by sweeping optimizer, learning rate, weight decay and architecture while following the project’s evaluation procedure (shuffle the dataset, split it 80/20, train on `data_training.csv`, predict `data_test.csv`). Over 24 random splits (best-of-3 restarts) the model scores a mean test binary cross-entropy of ≈ 0.083 , with roughly half of splits below 0.07. The residual loss is concentrated on a few samples (e.g. IDs 855133, 868202, 892189) that are labelled malignant but carry benign-looking features; whenever one lands in the 20% test split it adds a large loss that no reasonable hyperparameter choice can remove. The final model is the sigmoid $30 \rightarrow 24 \rightarrow 12 \rightarrow 2$ network of Section 7 trained with Adam ($\eta = 5 \cdot 10^{-4}$, weight decay 0.5, 250 epochs); the search scripts themselves are not part of the deliverables.

9 Code Structure and Size

The subject imposes no upper bound on code size; it only requires the three programs (data split, training, prediction) and a `Makefile`. This section reports the distribution of the submitted code in `src/`, measured in lines of code excluding all unit tests (both the inlined `#[cfg(test)]` modules and the integration tests in `tests/`). The full submission totals **3824** lines (3404 non-blank); the inlined unit tests add ~ 3325 lines and the integration tests in `tests/` another 530.

File	Functions	Lines
app/cli.rs	5	252
app/display.rs	6	152
app/mod.rs	0	6
app/predict.rs	1	65
app/split.rs	2	59
app/training.rs	4	256
app/types.rs	1	65
console.rs	4	36
data/loader.rs	3	95
data/mod.rs	0	4
data/preprocessing.rs	6	49
data/split.rs	7	101
lib.rs	0	24
main.rs	1	95
network/activation.rs	5	76
network/callbacks.rs	4	78
network/config.rs	13	159
network/initializer.rs	1	28
network/layer.rs	3	53
network/mod.rs	0	8
network/model.rs	14	175
network/persist.rs	4	131
training/backprop.rs	2	25
training/batch.rs	3	52
training/loss.rs	12	205
training/metrics.rs	5	106
training/mod.rs	0	8
training/monitor.rs	21	277
training/optimizer.rs	11	174
training/trainer.rs	15	285
visualization/mod.rs	0	1
visualization/plotter.rs	10	304
Total	163	3404

Table 1: Non-test lines of code per file in `src/` (blank lines excluded).

9.1 Per-file summary

9.2 Per-function breakdown

Lines counted for each function include its signature and body, grouped by file. The largest functions are `train_from_dataset` (187) and `parse_args` (162); together the training functions account for the bulk of the training logic.

A Notation Reference

Symbol	Meaning
$\mathbf{x}, \mathbf{y}, \mathbf{z}$	Column vectors
$\mathbf{W}$	Weight matrix
$\mathbf{b}$	Bias vector
B	Batch size
L	Number of layers
$n^{(\ell)}$	Number of neurons in layer ℓ
$\odot$	Element-wise (Hadamard) product
η	Learning rate
$\mathcal{L}$	Loss function
$\nabla_{\theta} \mathcal{L}$	Gradient of loss w.r.t. parameters θ

File	Function	Lines
app/cli.rs	default_dataset_path	3
	default_config_path	6
	usage	50
	parse_args	162
	apply_net_overrides	24
	activation_label	8
	initializer_label	7
	group_label	7
	print_verbose_config	99
	print_loaded_dataset	8
	print_loaded_config	20
	run_predict	56
	write_csv	13
	run_split	32
	build_dataset	23
	standardize_from_train	12
	timestamp_utc	22
	train_from_dataset	187
	default	14
	colors_enabled	3
	code	11
	paint	6
	bold	6
	read_dataframe	14
	column_to_values	28
	load_dataset	33
	fit	4
	transform	4
	fit_transform	4
	fit	4
	transform	4
	fit_transform	4
	split_and_collect_indices	10
	dataset_len	3
	select_dataset_rows	7
	stratify_keys	25
	rng_from_seed	9
	stratified_split	21
	train_test_split	19
	main	86
	forward	3
	backward	3
	backward	6
	as_function	13
	derivative	17
	new	3
	on_epoch_end	27
	on_train_begin	13
	on_batch_begin	10
	default_learning_rate	3
	default_epochs	3
	default_batch_size	3
	default_hidden_activation	3
	default_output_activation	3
	default_initializer	3
	default_activation_for	6
	activation_for	5
	from_yaml_file	6
	build_network	14
	resolved_layer_specs	15
	grouped_layers	16
	validate	29
	initialize	13